**UNIVERSIDAD DE CARABOBO**

**FACULTAD EXPERIMENTAL DE CIENCIAS Y TECNOLOGIA**

**NAGUANAGUA, CARABOBO**

# PRACTICA #4


**ESTUDIANTES:**

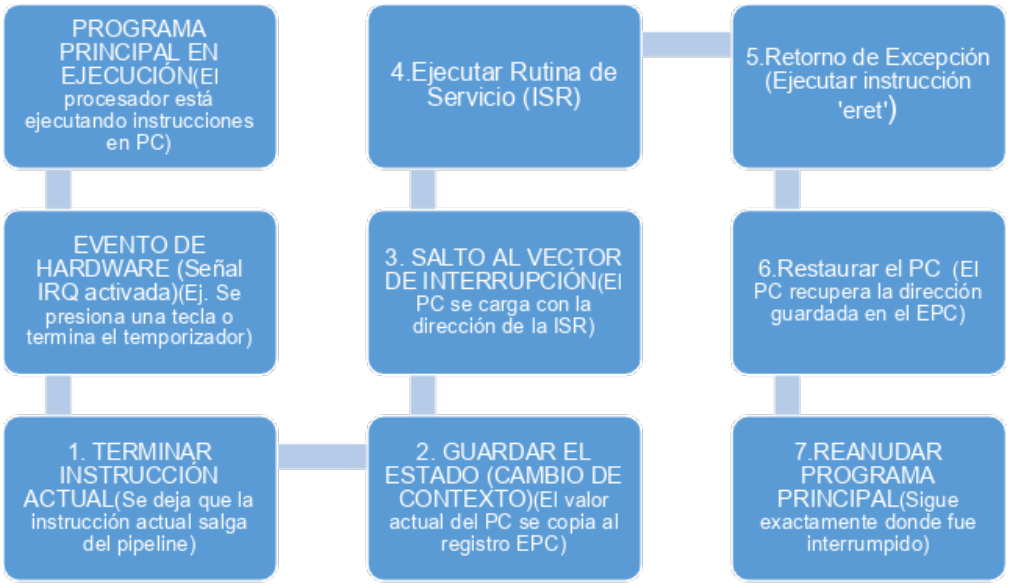
FABIAN PEÑA C.I: 31.994.692
SANTIAGO ESCALONA C.I: 31.768.925

**PROFESOR:**

JOSE CANACHE

# 1) Explique con un diagrama cómo funciona el ciclo de una interrupción de hardware (desde que ocurre el evento hasta que se atiende) .

Una interrupción de hardware es un mecanismo que permite a un dispositivo externo (como un teclado o un temporizador) pausar temporalmente la ejecución del programa principal para que el procesador atienda un evento urgente. El proceso funciona de la siguiente manera: Cuando el procesador recibe la señal de interrupción (IRQ), no se detiene de golpe, sino que permite que la instrucción actual termine su paso por el pipeline. Luego, para no "perderse", realiza un cambio de contexto: guarda la dirección de la siguiente instrucción que iba a ejecutar (copiando el valor del Program Counter o PC a un registro especial llamado EPC). Con su estado a salvo, el procesador salta a una dirección de memoria específica donde se encuentra la Rutina de Servicio de Interrupción (ISR). Esta rutina es el código encargado de atender al dispositivo que generó la alerta. Una vez que la rutina finaliza su tarea, se ejecuta una instrucción de retorno especial (como eret en MIPS).Esto hace que el procesador recupere la dirección que había guardado en el EPC y la devuelva al PC, permitiendo que el programa principal se reanude exactamente en el mismo punto donde fue interrumpido, como si nada hubiera pasado.



## 2) ¿Qué diferencias hay entre gestionar E/S con sondeo y hacerlo con interrupciones?

| **Característica** | **Gestión por Sondeo (Polling)** | **Gestión por Interrupciones** |
|---|---|---|
| Iniciativa de comunicación | El procesador pregunta constantemente al dispositivo si tiene datos listos o necesita atención | El dispositivo de E/S avisa activamente al procesador (mediante una señal de hardware) cuando requiere atención. |
| Eficiencia del Procesador | Baja. Se desperdician valiosos ciclos de reloj en esperas activas (busy waiting), leyendo repetidamente un registro de estado. | Alta. La CPU puede ejecutar otros programas o procesos mientras espera. Solo gasta ciclos cuando el evento realmente ocurre |
| Complejidad de Implementación | Baja (Software). Es muy fácil de programar. Solo requiere bucles de lectura condicional (justo como el código que hicimos para el Ejercicio 3). | Alta (Hardware y Software). Requiere hardware dedicado (pines IRQ, controlador de interrupciones) y programar Rutinas de Servicio (ISR) que guarden el contexto. |
| Tiempo de Respuesta | Variable/Lento. Depende de la frecuencia con la que el programa esté programado para revisar el estado del dispositivo. | Casi Inmediato. La alerta tiene prioridad y fuerza al procesador a atender el evento en cuanto termina la instrucción actual. |
| Caso de uso ideal | Dispositivos extremadamente rápidos, o sistemas integrados simples dedicados a una única tarea. | Sistemas operativos multitarea y dispositivos "lentos.eimpredecibles (como teclados, ratones o discos duros). |

Cuadro 1: Diferencias entre Sondeo e Interrupciones.

## 3) ¿Qué ventajas tiene el uso de interrupciones en términos de uso del procesador?

Maximización del tiempo útil (Eliminación del Busy Waiting): La principal ventaja es que libera al procesador de la .espera activa". En lugar de desperdiciar millones de ciclos de reloj

ejecutando bucles vacíos solo para verificar el estado de un periférico lento, la CPU utiliza ese tiempo para procesar otras instrucciones del programa principal.

Soporte fundamental para la Multitarea: Gracias a las interrupciones, los sistemas operativos pueden gestionar múltiples programas a la vez. Si un proceso necesita esperar a que el usuario presione una tecla o a que el disco duro lea un archivo, el sistema operativo le quita el control de la CPU y se lo da a otro proceso que sí esté listo para calcular, optimizando el uso del hardware al 100 %.

Gestión eficiente de la asincronía: Existe una brecha gigantesca entre la velocidad a la que opera la CPU (miles de millones de instrucciones por segundo) y la velocidad de los dispositivos de E/S (que dependen de la interacción humana o mecanismos físicos). Las interrupciones permiten sincronizar estos mundos sin que el componente más rápido deba frenarse a la velocidad del más lento.

Ahorro de energía (Eficiencia energética): En las arquitecturas modernas, si no hay tareas pendientes por ejecutar, las interrupciones permiten que el procesador entre en un estado de bajo consumo (modo idle o reposo). La CPU literalmente "duerme"para ahorrar batería y temperatura, y solo "despiertaçuando el pulso eléctrico de una interrupción de hardware se lo exige.

# 4) ¿Qué registros especiales se utilizan en MIPS32 para gestionar interrupciones?

Para gestionar las interrupciones y excepciones en la arquitectura MIPS32, no se utilizan los registros de propósito general convencionales. En su lugar, el procesador delega esta tarea al Coprocesador 0 (CP0), el cual cuenta con registros específicos dedicados al control del hardware y los privilegios del sistema. Los tres registros fundamentales para este proceso son:

**Registro Status (Registro 12 del CP0):** Actúa como el registro de control principal. Contiene el bit IE (Interrupt Enable), que determina si el procesador está habilitado para aceptar interrupciones de forma global (1) o si debe ignorarlas de forma absoluta (0). Además, gestiona los bits IM (Interrupt Mask), los cuales permiten enmascarar o desenmascarar líneas de interrupción específicas, decidiendo qué periféricos pueden interrumpir a la CPU y cuáles no. [cite: 36]

**Registro Cause (Registro 13 del CP0):** Es el encargado de identificar el origen y la naturaleza del evento. Almacena los bits IP (Interrupt Pending), que indican físicamente qué línea de hardware específica solicitó la atención del procesador. [cite: 38] También incluye el campo ExcCode (Exception Code), el cual clasifica el tipo de evento ocurrido (por ejemplo, una interrupción externa de hardware, un error aritmético o una llamada al sistema).

**Registro EPC (Registro 14 del CP0 - Exception Program Counter):** Funciona como el registro de salvaguarda del contexto. Cuando ocurre una interrupción, el hardware guarda automáticamente en el EPC la dirección de memoria de la instrucción que fue interrumpida en la ruta de datos (datapath). Esto garantiza que, al finalizar la Rutina de Servicio de Interrupción (ISR), la instrucción especial de retorno (eret) pueda restaurar el contador de

programa (PC) y reanudar la ejecución del programa principal exactamente donde se detuvo. Dado que estos registros pertenecen al Coprocesador 0, no son accesibles mediante las instrucciones regulares del procesador. Para interactuar con ellos, el conjunto de instrucciones de MIPS proporciona instrucciones privilegiadas específicas: mfc0 (Move From Coprocessor 0) para la lectura de su contenido, y mtc0 (Move To Coprocessor 0) para su escritura.

# 5) ¿Por qué es necesario guardar el contexto (registros) al entrar en una rutina de servicio?

Guardar el contexto (el estado actual de los registros de la CPU) al inicio de una Rutina de Servicio de Interrupción (ISR) es un paso crítico para garantizar la integridad del sistema y la correcta reanudación del programa principal. Esto es estrictamente necesario por las siguientes razones técnicas:

**Recurso de hardware compartido:** La arquitectura del procesador (como en el caso de MIPS32) posee un único banco de registros de propósito general (como los registros $t0-$t9, $s0-$s7, $a0-$a3, etc.). [cite: 48] Tanto el programa principal que se estaba ejecutando como la nueva rutina de interrupción dependen de este mismo banco físico para realizar cualquier operación lógica, aritmética o de carga de memoria.

**Prevención de corrupción de datos:** Al momento de atender la interrupción, el código de la ISR inevitablemente necesitará utilizar algunos de estos registros para procesar el evento (por ejemplo, cargar la dirección de memoria del teclado o evaluar una condición). Si la ISR sobrescribe el valor de un registro que el programa principal estaba utilizando activamente, al retornar de la interrupción, el programa principal operará con datos corruptos. Esto produce cálculos erróneos, bucles infinitos o el colapso total del programa.

**Principio de transparencia:** Las interrupciones por hardware son eventos asíncronos;el programa principal "no sabe.^en^ qué ciclo de reloj exacto van a ocurrir. [cite: 54] Por reglas de diseño en arquitectura de computadores, la ejecución de una ISR debe ser completamente ïnvisible"para el programa que fue interrumpido. Para lograr esta transparencia, la ISR está obligada a dejar la CPU en el estado idéntico en el que la encontró.

# 6) Momentos en que pueden generarse excepciones en un sistema MIPS32.

**a) Enumera al menos 4 situaciones en las que se pueda generar una excepción** (por ejemplo: desbordamiento aritmético, fallo de dirección, etc.).

- **Desbordamiento Aritmético:** Ocurre cuando el resultado de una operación matemática con signo (utilizando instrucciones como add o addi) excede la capacidad de

representación de los 32 bits del registro. (Nota: las instrucciones sin signo como addu ignoran este desbordamiento y no generan excepción).

- **Error de Dirección:** Se produce al intentar acceder a la memoria de forma incorrecta. El caso más común en MIPS es intentar cargar una palabra entera (lw) o guardarla (sw) en una dirección de memoria que no es múltiplo de 4 (acceso desalineado).
- **Llamada al Sistema Intencional (System Call):** Es una excepción generada por software de manera voluntaria. Ocurre cuando el programa ejecuta la instrucción syscall (como las que usamos en el semáforo para hacer el Sleep o imprimir texto), lo que le cede el control temporalmente al sistema operativo.
- **Instrucción Ilegal o Reservada:** Sucede cuando la CPU lee de la memoria de instrucciones un código (opcode) que no existe o no es válido dentro del repertorio de instrucciones de MIPS32. **b) Explica qué etapas del pipeline pueden provocar una excepción y por qué.** [cite: 66]

  1. **Etapa IF (Búsqueda de Instrucción / Instruction Fetch): Qué excepción provoca:** Error de dirección de instrucción o fallo de página. [cite: 68] **Por qué:** Ocurre si el Program Counter (PC) apunta a una dirección de memoria que está mal alineada (no termina en 00 en binario) o intenta leer una instrucción de un área de memoria protegida por el sistema operativo.
  2. **Etapa ID (Decodificación / Instruction Decode): Qué excepción provoca:** Instrucción ilegal / no definida, o la ejecución de un syscall. **Por qué:** Es en esta fase donde la Unidad de Control lee los bits del opcode. [cite: 72] Si la combinación de bits no corresponde a ninguna instrucción válida en el hardware, se levanta la bandera de error antes de intentar ejecutarla.
  3. **Etapa EX (Ejecución / Execute): Qué excepción provoca:** Desbordamiento aritmético (Overflow). **Por qué:** Aquí es donde la ALU (Unidad Aritmético Lógica) realiza el cálculo real. Si el hardware de la ALU detecta que los bits de acarreo indican un desbordamiento en una operación con signo, dispara la excepción inmediatamente.
  4. **Etapa MEM (Acceso a Memoria / Memory Access): Qué excepción provoca:** Error de dirección de datos. **Por qué:** Es la etapa donde se ejecutan los accesos a la memoria RAM (lw, sw). Si la dirección efectiva calculada por la ALU en la etapa anterior resulta estar mal alineada para el tamaño del dato, el controlador de memoria rechaza la operación y genera la excepción.

     La etapa WB (Write Back / Escritura) nunca genera excepciones. Si una instrucción logra llegar a esta etapa (WB), el procesador asume que es 100 % segura y procede a alterar el estado arquitectónico (escribiendo el resultado en el banco de registros). ] Cualquier problema debió ser detectado en las 4 etapas previas.

# 7) Estrategias de tratamiento de excepciones e interrupciones

**a) Diferencias entre interrupciones y excepciones:**

- **Interrupciones (Asíncronas):** Son eventos externos al procesador provocados por periféricos (teclado, disco duro, temporizador). El programa no sabe cuándo ocurrirán.
- **Excepciones (Síncronas):** Son eventos internos generados por la propia ejecución de una instrucción, como un desbordamiento aritmético o una división por cero.

**b) Estrategias y registro EPC:** Cuando ocurre un evento, el procesador detiene el flujo normal y salta a una dirección fija de memoria donde reside la Rutina de Servicio (ISR). **Registro EPC (Exception Program Counter):** Es fundamental porque guarda automáticamente la dirección de la instrucción que fue interrumpida. Al finalizar la rutina, la instrucción eret usa el valor del EPC para devolver el control al programa principal exactamente donde se detuvo.

# 8) Habilitación de interrupciones en dispositivos y procesador

**a) Habilitación:**

- **Teclado/Pantalla:** Se habilitan poniendo en '1' el bit de interrupción en sus respectivos registros de control (mapeados en memoria).
- **Procesador (Registro Status):** Se debe modificar el bit IE (Interrupt Enable), que es el interruptor global de interrupciones del sistema. También se deben configurar los bits IM (Interrupt Mask) para decidir qué líneas de hardware específicas tienen permiso para interrumpir.

**b) ¿Qué pasaría si solo se habilitan en el dispositivo?:** El dispositivo enviará la señal eléctrica (IRQ), pero el procesador la ignorará por completo si el bit IE del registro Status está en 0. La interrupción quedará como "pendiente.^en el registro Cause, pero nunca será atendida. [cite: 100]

# 9) Procesamiento de interrupciones (Paso a paso)

**a) Ciclo de una interrupción de reloj:**

- **Evento:** El temporizador llega a su límite y envía una señal IRQ.

- **Hardware:** La CPU termina la instrucción actual, guarda el PC en el EPC y salta a la dirección de la ISR.
- **Software (ISR):** Se guarda el contexto (registros de propósito general como $t0, $s0, etc.) en la pila.
- **Atención:** La rutina procesa el evento de reloj (ej. actualizar el contador de tiempo).
- **Retorno:** Se restaura el contexto de la pila y se ejecuta eret para volver al programa original.

**b) Importancia de guardar el contexto:** Es vital porque el programa principal y la rutina de servicio comparten el mismo banco físico de registros. Si la rutina no guarda los valores originales, sobrescribirá datos activos del programa, causando cálculos erróneos o colapsos al intentar retomar la ejecución.

# 10) Interrupciones de reloj y control de ejecución

**a) Aplicaciones:**

- **Evitar bucles infinitos:** El sistema operativo programa una interrupción de reloj periódica. Si un programa queda atrapado en un bucle sin fin, la interrupción de reloj permite al SO retomar el control de la CPU para finalizar el proceso.
- **Tiempo máximo de ejecución:** Se asigna un "quantum"de tiempo a cada programa. Al expirar el tiempo (detectado por el reloj), se genera una excepción que permite al planificador del sistema operativo pasar al siguiente proceso.

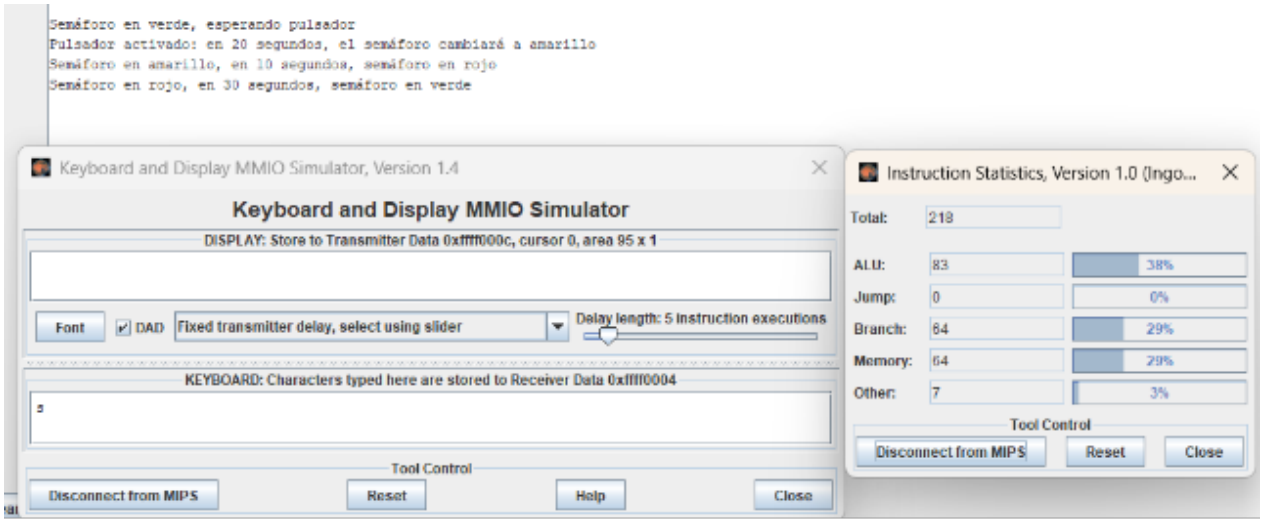
**b) Finalización antes de la interrupción:** Si el programa termina exitosamente antes de que ocurra la interrupción de reloj, el software debe cancelar o resetear el temporizador para evitar que la CPU salte a una rutina de servicio innecesaria que ya no tiene proceso que atender.

# 11) Análisis y Discusión de los Resultados

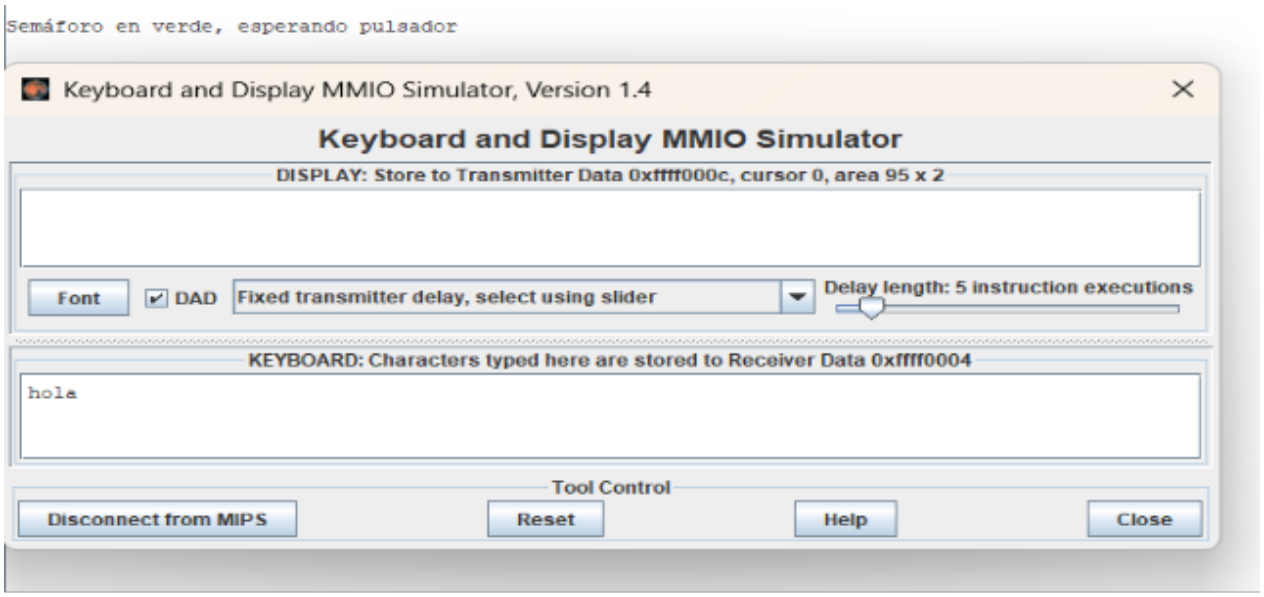
Primero analizemos los casos del Semáforo con Pulsador

**1. Mejor Caso:** Ocurre cuando el usuario presiona la tecla 's' o 'S' casi inmediatamente después de que el programa inicia y muestra el mensaje de "Semáforo en verde". El procesador entra al bucle de sondeo (esperar_tecla), lee el registro de control del teclado (0xffff0000) y detecta rápidamente que el bit de Ready es 1. Al leer el dato, la comparación (beq) coincide inmediatamente con el código

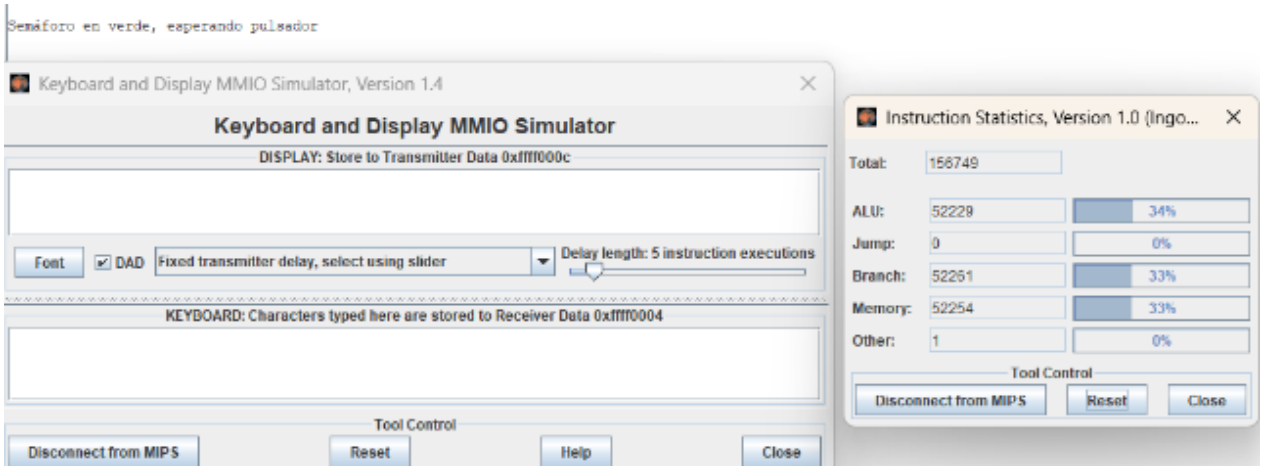
ASCII correcto. Este es el escenario ideal porque el procesador pasa el mínimo tiempo posible atrapado en el bucle de .espera activa"(busy waiting). El programa avanza rápidamente a su función principal (la máquina de estados del semáforo y las llamadas al sistema para pausar el tiempo), optimizando el uso de la CPU. [




**2. Caso de error:** El usuario presiona teclas incorrectas (por ejemplo, números, la barra espaciadora, o letras como la 'a' o 'x') repetidas veces mientras el semáforo está en verde. El teclado genera el evento y el procesador lee el dato en la dirección 0xffff0004. Sin embargo, las instrucciones lógicas comparan el valor ASCII ingresado con 115 ('s') y 83 ('S'). [cite: 127] Como ninguna condición se cumple, el programa ignora el dato, ejecuta el salto incondicional (j esperar_tecla) y limpia efectivamente el buffer visual, volviendo a sondear. El programa demuestra ser robusto. No colapsa, no arroja excepciones de memoria y no avanza al estado amarillo por accidente. Este caso comprueba que la lógica de filtrado implementada en ensamblador funciona correctamente, descartando cualquier ruido.º error de capa de usuario.




**3. Peor caso:** El semáforo inicia en verde, pero el usuario nunca presiona el pulsador (la tecla 's'), o tarda horas en hacerlo. La CPU queda atrapada en un bucle infinito evaluando constantemente la misma dirección de memoria (0xffff0000) millones de veces por segundo, encontrando siempre un 0.Desde el punto de vista de la arquitectura, este es el peor caso porque evidencia la mayor debilidad técnica del método de sondeo (polling).El procesador alcanza un 100 % de uso de CPU realizando una tarea completamente inútil (preguntar ¿a presionaste?", ¿a presionaste?"). Esto consume energía y bloquea la ejecución de cualquier otra tarea, demostrando en la práctica por qué los sistemas modernos prefieren utilizar el ciclo de interrupciones por hardware.




La cantidad total de instrucciones sigue subiendo y subiendo ya que esta atrapada en un "bucle infinito".

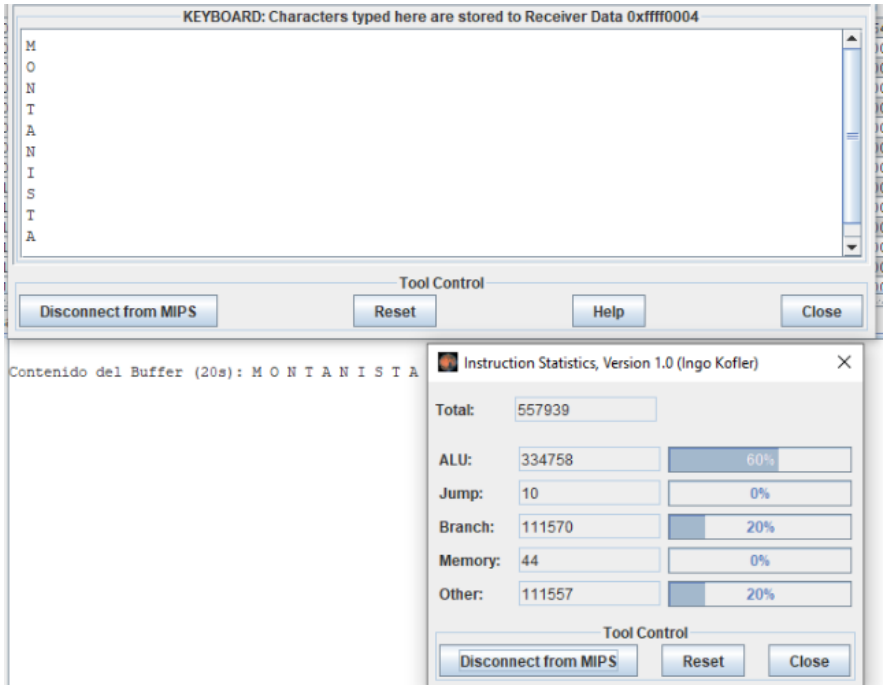
Ahora analizamos los casos del Buffer Circular con Interrupciones

El programa P4BufferA.asm tiene como objetivo capturar caracteres de forma asíncrona utilizando el sistema de interrupciones de MIPS32. A diferencia del programa de sondeo(Polling), el procesador no se queda bloqueado esperando al usuario; en su lugar, el main ejecuta un ciclo de espera de 20 segundos mientras el hardware del teclado "despierta.ªl procesador solo cuando hay un dato listo. La razón de ser de este diseño es la eficiencia: permite que la CPU realice tareas (como llevar la cuenta del tiempo) y solo atienda al periférico cuando ocurre un evento real, Además, implementa un buffer circular que garantiza que el sistema no colapse si se reciben más datos de los esperados.

**Análisis de Casos de Ejecución**

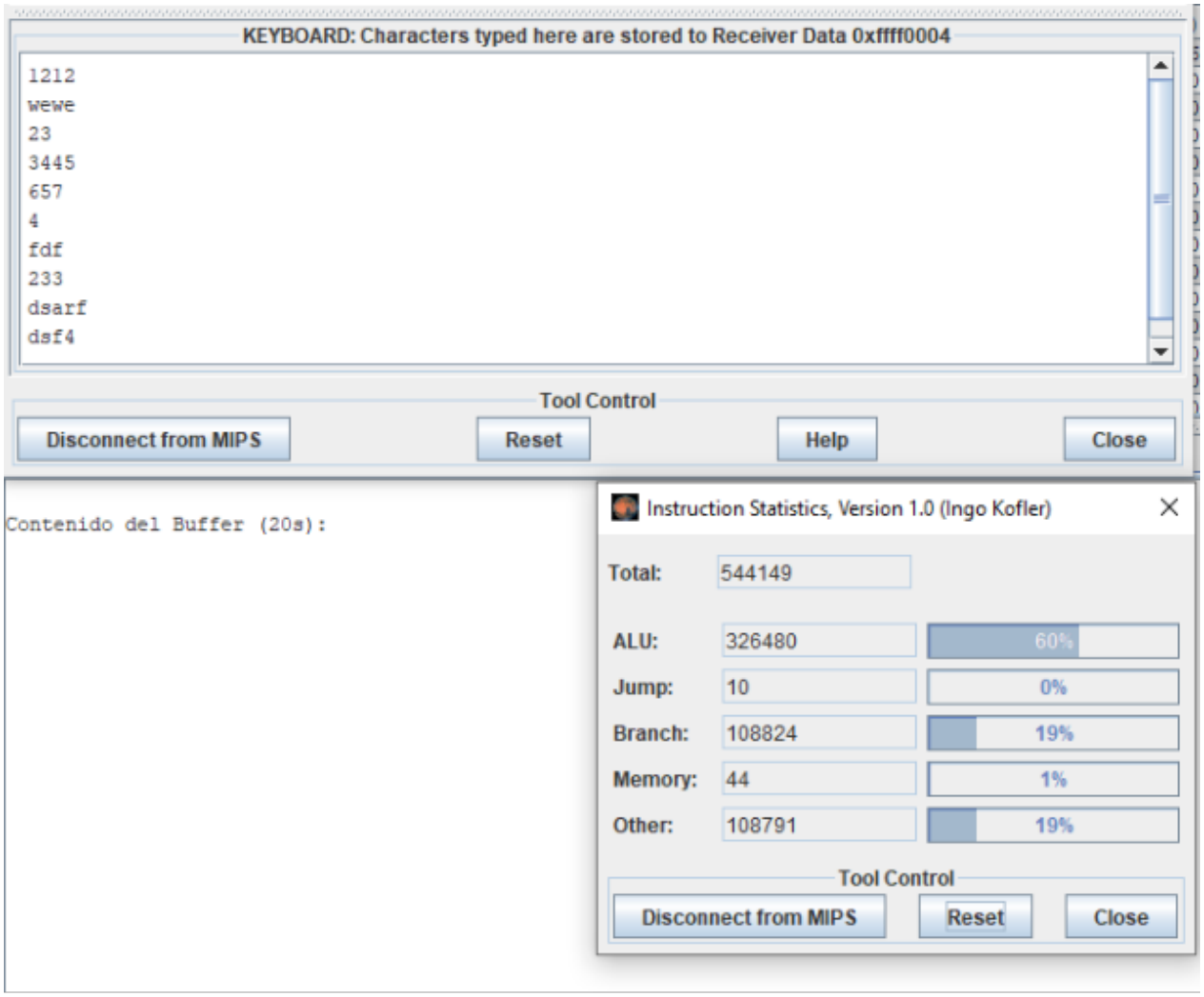
En este tipo de arquitecturas orientadas a interrupciones, los çasos"no se definen por el éxito o fracaso del programa (ya que el código es robusto y no colapsa), sino por la optimización de los recursos y la integridad de la información:

**Mejor Caso (Sincronía y Eficiencia):** Ocurre cuando el usuario ingresa una secuencia de hasta 10 caracteres válidos (A-Z). En este escenario, cada interrupción de hardware se traduce en un dato útil almacenado en el segmento .data.El procesador realiza un cambio de contexto mínimo y eficiente, cumpliendo el objetivo de captura al 100 % sin desperdiciar ciclos de reloj ni sobrescribir información.

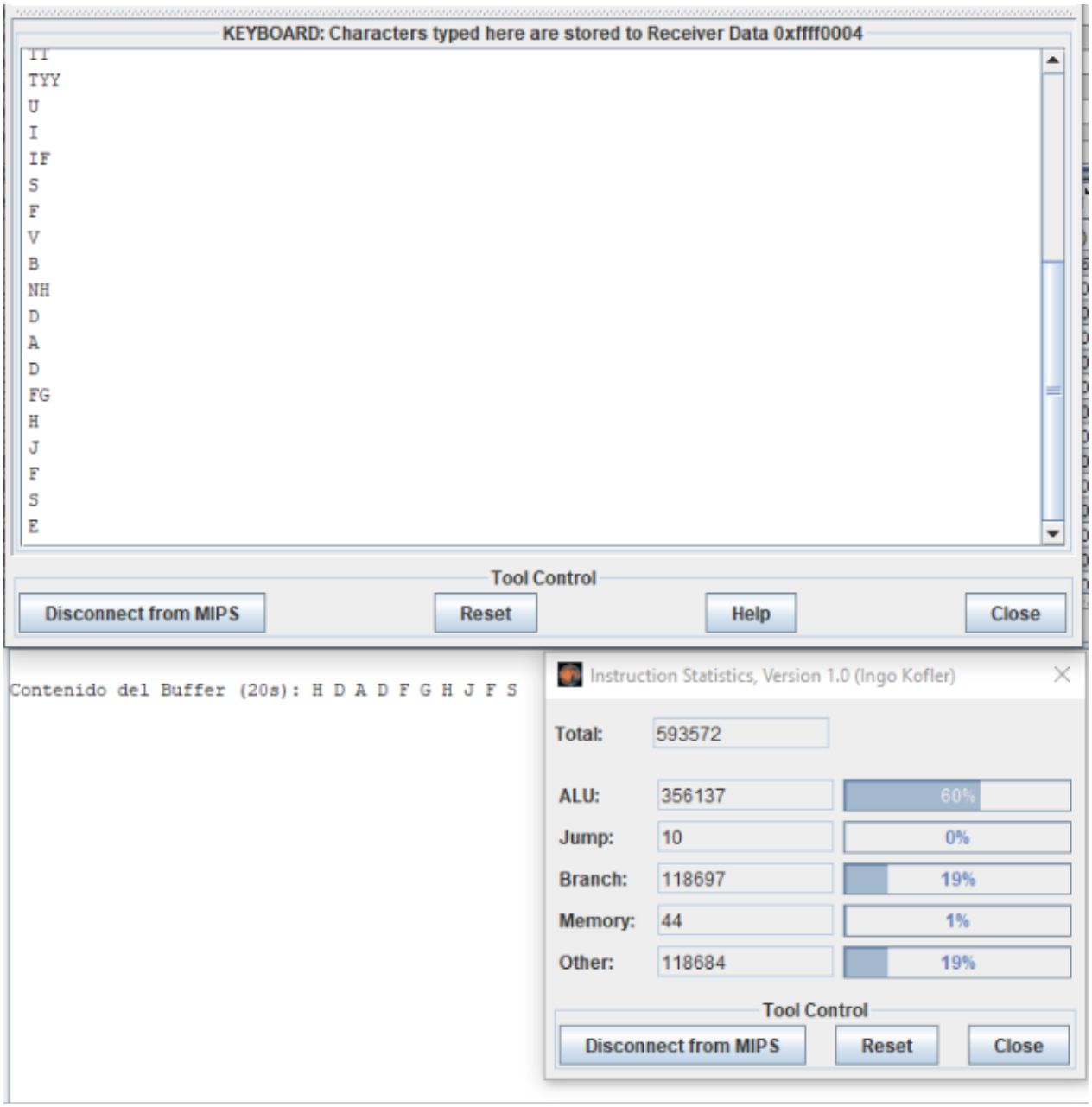



**Caso de Error de Entrada (Resiliencia del Filtro):** Se presenta cuando el usuario presiona teclas que no pertenecen al rango permitido (números o minúsculas).Aunque el hardware genera la interrupción y obliga a la CPU a entrar en modo Kernel, la lógica de filtrado ASCII actúa como un cortafuego. El .error"(la entrada inválida) es gestionado preventivamente: el sistema gasta ciclos en proce-

sar la interrupción, pero protege la integridad del buffer al no permitir la entrada de ruido".




**Peor Caso (Saturación de Datos y Pérdida de Historial):** Ocurre cuando el flujo de entrada supera la capacidad física del buffer (más de 10 pulsaciones válidas). [cite: 151] Debido a la naturaleza circular del algoritmo, el puntero regresa a la dirección base y comienza a escribir sobre los primeros datos. Análisis Técnico: Este es el "peor caso"porque, aunque el sistema se mantiene perfectamente estable y no arroja excepciones de memoria (Out of Bounds), se pierde la trazabilidad de los datos antiguos. El algoritmo prioriza la disponibilidad del sistema (seguir funcionando) sobre la persistencia total de la información, aunque este problema es solucionado agrandando la capacidad del buffer.




Una característica crítica de este desarrollo es que el tiempo de ejecución total es una variable dependiente del usuario. Al estar estructurado como un bucle que se repite cada 20 segundos, el programa puede funcionar durante minutos u horas sin degradar su rendimiento. Esta naturaleza indefinida demuestra la superioridad de las interrupciones sobre el sondeo: el procesador no se desgasta .esperando", sino que simplemente responde a la demanda. En conclusión, la robustez de P4BufferA.asm reside en su capacidad de transformar un escenario de tiempo indeterminado y entradas impredecibles en un proceso controlado, seguro y eficiente, donde los límites físicos (el tamaño del buffer) son gestionados por software para evitar cualquier excepción de memoria.